



HCI – intelligent *multimodal* interfaces

Deep feature matching

Master program in Artificial Intelligence

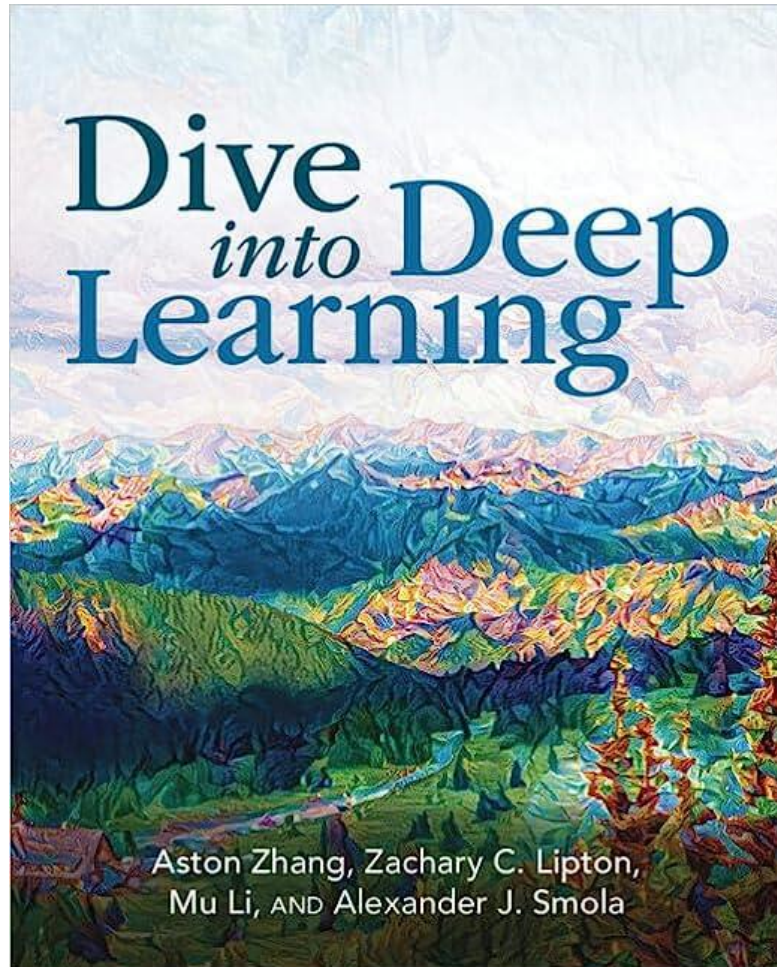
Motivation

- computer vision and in particular **multiple view geometry** consists of theorems and algorithms built on the assumption that **interest points** can be reliably **extracted** and **matched** across images.



However, the inputs to most real-world computer vision systems are raw images, not idealized point locations

Deep learning



<https://d2l.ai/>

1. Introduction

Preview Version

PyTorch

MXNet

Notebooks

Courses

GitHub

1. Introduction

Until recently, nearly every computer program that you might have interacted with during an ordinary day was coded up as a rigid set of rules specifying precisely how it should behave. Say that we wanted to write an application to manage an e-commerce platform. After huddling around a whiteboard for a few hours to ponder the problem, we might settle on the broad strokes of a working solution, for example: (i) users interact with the application through an interface running in a web browser or mobile application; (ii) our application interacts with a commercial-grade database engine to keep track of each user's state and maintain records of historical transactions; and (iii) at the heart of our application, the *business logic* (you might say, the *brains*) of our application spells out a set of rules that map every conceivable circumstance to the corresponding action that our program should take.

To build the brains of our application, we might enumerate all the common events that our program should handle. For example, whenever a customer clicks to add an item to their shopping cart, our program should add an entry to the shopping cart database table, associating that user's ID with the requested product's ID. We might then attempt to step

Deep learning

- Traditional approach:
 - A large dataset with ground truth is required for supervised learning procedures



Supervised vs unsupervised

Training strategy

- training neural networks with strong supervision of interest points is non-trivial



where a network is trained to detect body parts such as the corner of the mouth or left ankle, the notion of interest point detection is semantically ill-defined

Superpoint

- produces interest point detections
- Produces fixed length descriptors in a single forward pass

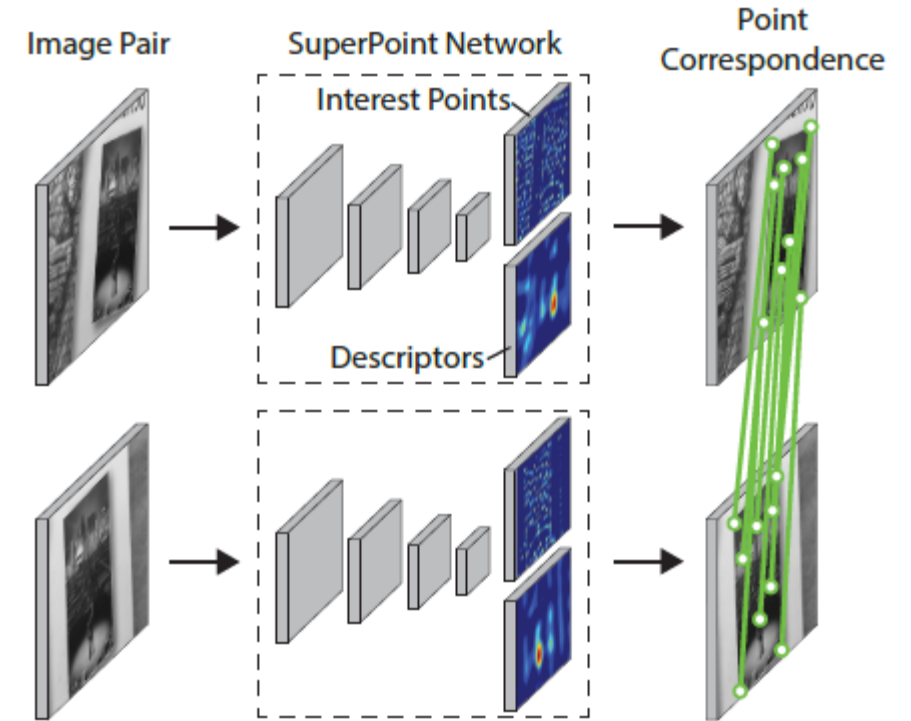


Figure 1. **SuperPoint for Geometric Correspondences.** We present a fully-convolutional neural network that computes SIFT-like 2D interest point locations and descriptors in a single forward pass and runs at 70 FPS on 480×640 images with a Titan X GPU.

Superpoint

- Training on Magic point,
- Homographic adaptation

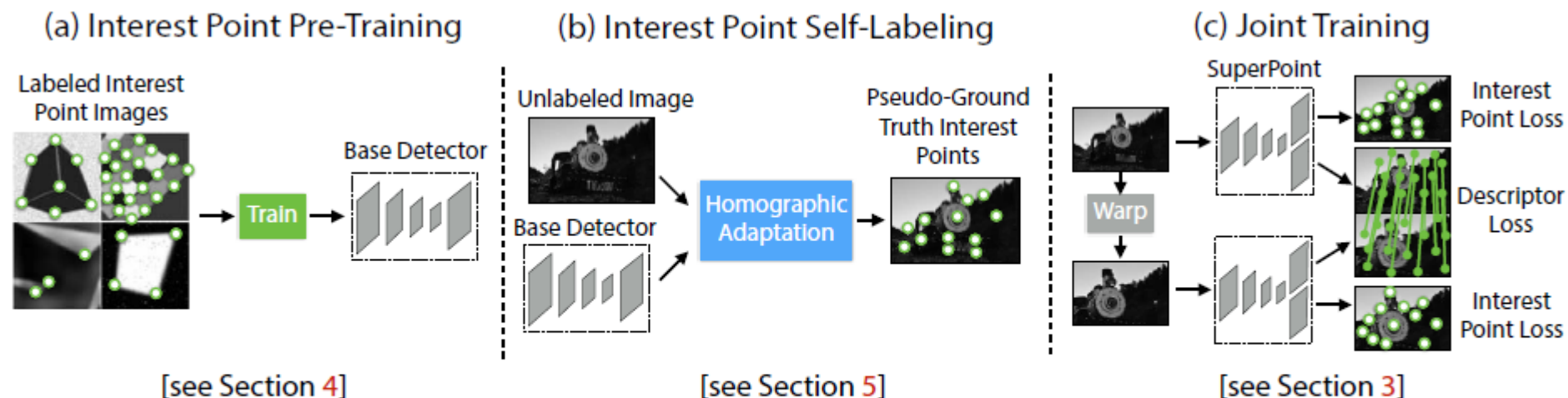
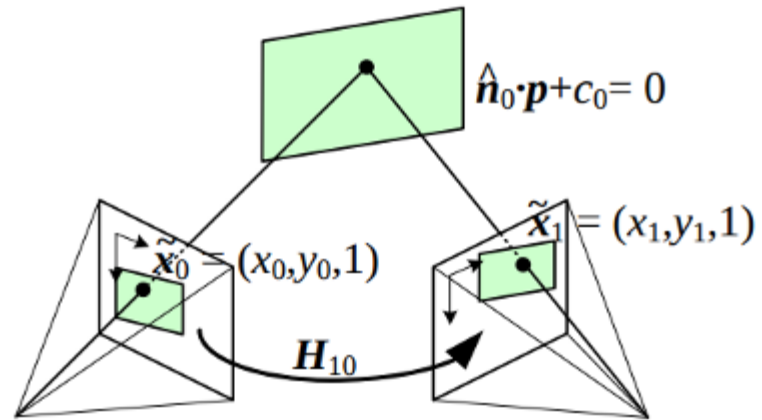
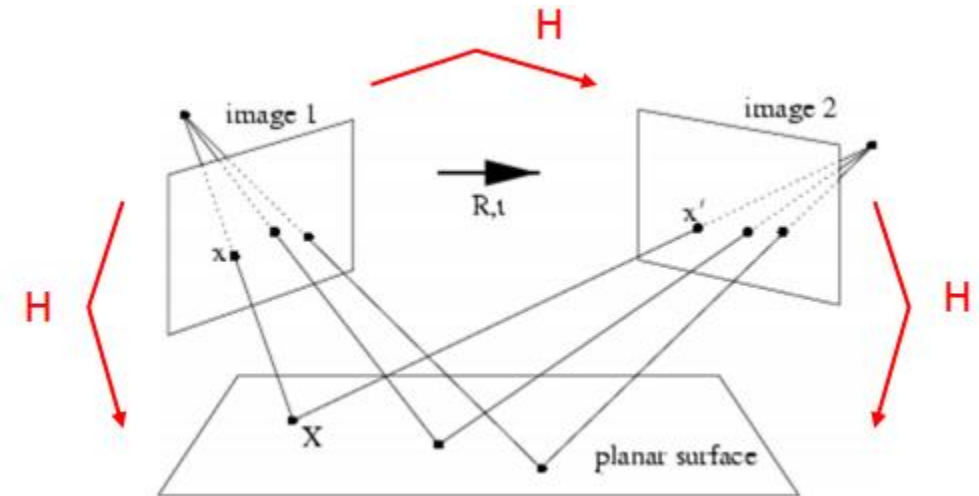
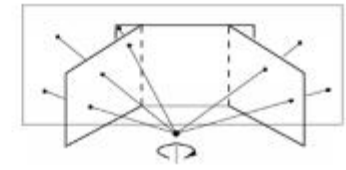
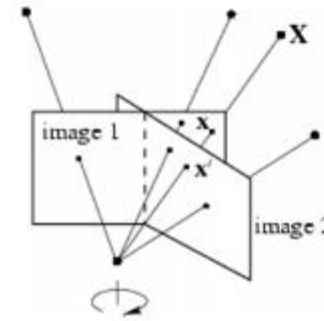


Figure 2. **Self-Supervised Training Overview.** In our self-supervised approach, we (a) pre-train an initial interest point detector on synthetic data and (b) apply a novel Homographic Adaptation procedure to automatically label images from a target, unlabeled domain. The generated labels are used to (c) train a fully-convolutional network that jointly extracts interest points and descriptors from an image.

Homography



Rotating camera, arbitrary world



<https://answers.opencv.org/answers/141414/revisions/>

[https://en.wikipedia.org/wiki/Homography_\(computer_vision\)](https://en.wikipedia.org/wiki/Homography_(computer_vision))

Superpoint NN architecture

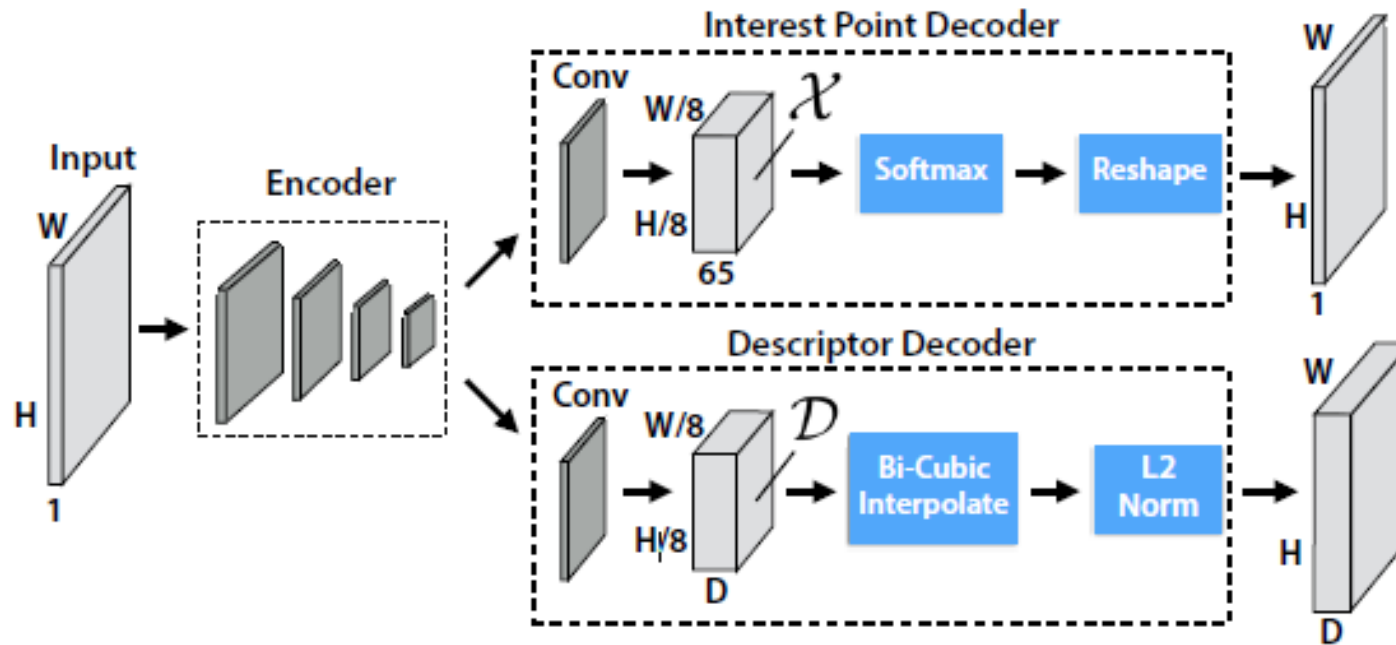


Figure 3. **SuperPoint Decoders.** Both decoders operate on a shared and spatially reduced representation of the input. To keep the model fast and easy to train, both decoders use non-learned upsampling to bring the representation back to $\mathbb{R}^{H \times W}$.

Superpoint NN architecture

The model has a single, shared encoder to process and reduce the input image dimensionality,

- convolutional layers,
- spatial downsampling via pooling
- non-linear activation functions.



The encoder maps the input image $I \in \mathbb{R}^{H \times W}$ to an intermediate tensor $\mathcal{B} \in \mathbb{R}^{H_c \times W_c \times F}$ with smaller spatial dimension and greater channel depth (i.e., $H_c < H$, $W_c < W$ and $F > 1$).



7. Convolutional Neural Networks

Image data is represented as a two-dimensional grid of pixels, be the image monochromatic or in color. Accordingly each pixel corresponds to one or multiple numerical values respectively. So far we have ignored this rich structure and treated images as vectors of numbers by *flattening* them, irrespective of the spatial relation between pixels. This deeply unsatisfying approach was necessary in order to feed the resulting one-dimensional vectors through a fully connected MLP.

Because these networks are invariant to the order of the features, we could get similar results regardless of whether we preserve an order corresponding to the spatial structure of the pixels or if we permute the columns of our design matrix before fitting the MLP's parameters. Ideally, we would leverage our prior knowledge that nearby pixels are typically related to each other, to build efficient models for learning from image data.

This chapter introduces *convolutional neural networks* (CNNs) ([LeCun et al., 1995](#)), a powerful family of neural networks

7.5. Pooling

[SAGEMAKER STUDIO LAB](#)

[COLAB \[PYTORCH\]](#)

In many cases our ultimate task asks some global question about the image, e.g., *does it contain a cat?* Consequently, the units of our final layer should be sensitive to the entire input. By gradually aggregating information, yielding coarser and coarser maps, we accomplish this goal of ultimately learning a global representation, while keeping all of the advantages of convolutional layers at the intermediate layers of processing. The deeper we go in the network, the larger the receptive field (relative to the input) to which each hidden node is sensitive. Reducing spatial resolution accelerates this process, since the convolution kernels cover a larger effective area.

Moreover, when detecting lower-level features, such as edges (as discussed in [Section 7.2](#)), we often want our representations to be somewhat invariant to translation. For instance, if we take the image X with a sharp delineation between black and white and shift the whole image by one pixel to the right, i.e., $Z[i, j] = X[i, j + 1]$, then the output for the new image Z might be vastly different. The edge will have shifted by one pixel. In reality, objects hardly ever occur exactly at the same place. In fact, even with a tripod and a stationary object, vibration of the camera due to the movement of the shutter might shift everything by a pixel or so (high-end cameras are loaded with special features to address this problem).

5.1.2. Activation Functions

Activation functions decide whether a neuron should be activated or not by calculating the weighted sum and further adding bias to it. They are differentiable operators for transforming input signals to outputs, while most of them add nonlinearity. Because activation functions are fundamental to deep learning, let's briefly survey some common ones.

5.1.2.1. ReLU Function

The most popular choice, due to both simplicity of implementation and its good performance on a variety of predictive tasks, is the *rectified linear unit* (*ReLU*) ([Nair and Hinton, 2010](#)). ReLU provides a very simple nonlinear transformation. Given an element x , the function is defined as the maximum of that element and 0:

$$\text{ReLU}(x) = \max(x, 0). \tag{5.1.4}$$

Informally, the ReLU function retains only positive elements and discards all negative elements by setting the corresponding activations to 0. To gain some intuition, we can plot the function. As you can see, the activation function is piecewise linear.

Superpoint NN architecture

After the encoder, the architecture splits into two decoder “heads”, which learn task specific weights – one for interest point detection and the other for interest point description.

10.6. The Encoder–Decoder Architecture

[COLAB \[PYTORCH\]](#)

[SAGEMAKER STUDIO LAB](#)

In general sequence-to-sequence problems like machine translation ([Section 10.5](#)), inputs and outputs are of varying lengths that are unaligned. The standard approach to handling this sort of data is to design an *encoder–decoder* architecture ([Fig. 10.6.1](#)) consisting of two major components: an *encoder* that takes a variable-length sequence as input, and a *decoder* that acts as a conditional language model, taking in the encoded input and the leftwards context of the target sequence and predicting the subsequent token in the target sequence.

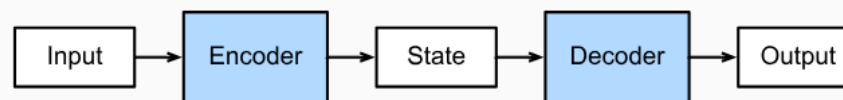


Fig. 10.6.1 The encoder–decoder architecture.

Superpoint NN architecture

For **interest point detection**, each pixel of the output corresponds to a probability of “point-ness” for that pixel in the input.

The standard network design for dense prediction involves an **encoder-decoder** pair, where the spatial resolution is decreased via pooling or strided convolution, and then **upsampled** back to full resolution via upconvolution operations

Superpoint NN architecture

- The **descriptor** head computes $D \times 2 \times R \times H_c \times W_c \times D$ and outputs a tensor sized $R \times H \times W \times D$
- To output a dense map of L2-normalized fixed length descriptors, we use a model to first output a semi-dense grid of descriptors (e.g., one every 8 pixels)
- The decoder then performs **bicubic interpolation** of the descriptor and then **L2-normalizes** the activations to be unit length.

Superpoint

- Loss function

$$\mathcal{L}(\mathcal{X}, \mathcal{X}', \mathcal{D}, \mathcal{D}'; Y, Y', S) = \mathcal{L}_p(\mathcal{X}, Y) + \mathcal{L}_p(\mathcal{X}', Y') + \lambda \mathcal{L}_d(\mathcal{D}, \mathcal{D}', S). \quad (1)$$

Image Matching WebUI

Identify matching points between two images

Description

This simple tool efficiently matches image pairs using multiple famous image matching algorithms. The tool features a Graphical User Interface (GUI) designed using [gradio](#). You can effortlessly select two images and a matching algorithm and obtain a precise matching result. **Note:** the images source can be either local images or webcam images.

Try it on



Hugging Face

Spaces



Open in Studio

<https://github.com/Vincentqyw/image-matching-webui?tab=readme-ov-file>

- ☒ [EfficientLoFTR](#), CVPR 2024
- ☒ [MASt3R](#), CVPR 2024
- ☒ [DUST3R](#), CVPR 2024
- ☒ [OmniGlue](#), CVPR 2024
- ☒ [XFeat](#), CVPR 2024
- ☒ [RoMa](#), CVPR 2024
- ☒ [DeDoDe](#), 3DV 2024
- ☐ [Mickey](#), CVPR 2024
- ☒ [GIM](#), ICLR 2024
- ☐ [DUST3R](#), arXiv 2023
- ☒ [LightGlue](#), ICCV 2023
- ☒ [DarkFeat](#), AAAI 2023
- ☒ [SFD2](#), CVPR 2023
- ☒ [IMP](#), CVPR 2023
- ☐ [ASTR](#), CVPR 2023
- ☐ [SEM](#), CVPR 2023
- ☐ [DeepLSD](#), CVPR 2023
- ☒ [GlueStick](#), ICCV 2023
- ☐ [ConvMatch](#), AAAI 2023
- ☒ [LoFTR](#), CVPR 2021
- ☒ [SOLD2](#), CVPR 2021



<https://huggingface.co/spaces/Realcat/image-matching-webui>



+ Codice + Testo Copia su Drive



▼ LightGlue Demo



In this notebook we match two pairs of images using LightGlue with early stopping and point pruning.



```
[ ] # If we are on colab: this clones the repo and installs the dependencies
    from pathlib import Path

    if Path.cwd().name != "LightGlue":
        !git clone --quiet https://github.com/cvg/LightGlue/
        %cd LightGlue
        !pip install --progress-bar off --quiet -e .

    from lightglue import LightGlue, SuperPoint, DISK
    from lightglue.utils import load_image, rbd
    from lightglue import viz2d
    from scipy import spatial
    import numpy as np
    import torch

    torch.set_grad_enabled(False)
    images = Path("assets")
```


SuperPoint: Self-Supervised Interest Point Detection and Description

Daniel DeTone
Magic Leap
Sunnyvale, CA

ddetone@magicleap.com

Tomasz Malisiewicz
Magic Leap
Sunnyvale, CA

tmalisiewicz@magicleap.com

Andrew Rabinovich
Magic Leap
Sunnyvale, CA

arabinovich@magicleap.com

Abstract

This paper presents a self-supervised framework for training interest point detectors and descriptors suitable for a large number of multiple-view geometry problems in computer vision. As opposed to patch-based neural networks, our fully-convolutional model operates on full-sized images and jointly computes pixel-level interest point locations and associated descriptors in one forward pass. We introduce Homographic Adaptation, a multi-scale, multi-homography approach for boosting interest point detection repeatability and performing cross-domain adaptation (e.g., synthetic-to-real). Our model, when trained on the MS-COCO generic image dataset using Homographic Adaptation, is able to repeatedly detect a much richer set of interest points than the initial pre-adapted deep model and any other traditional corner detector. The final system gives rise to state-of-the-art homography estimation results on HPatches when compared to LIFT, SIFT and ORB.

1. Introduction

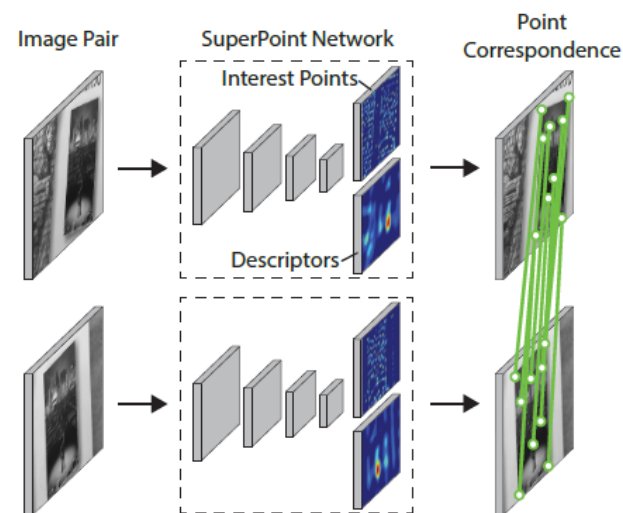


Figure 1. **SuperPoint for Geometric Correspondences.** We present a fully-convolutional neural network that computes SIFT-like 2D interest point locations and descriptors in a single forward pass and runs at 70 FPS on 480×640 images with a Titan X GPU.

<https://github.com/rpautrat/SuperPoint>

SuperGlue: Learning Feature Matching with Graph Neural Networks

Paul-Edouard Sarlin^{1*} Daniel DeTone² Tomasz Malisiewicz² Andrew Rabinovich²
¹ ETH Zurich ² Magic Leap, Inc.

Abstract

This paper introduces SuperGlue, a neural network that matches two sets of local features by jointly finding correspondences and rejecting non-matchable points. Assignments are estimated by solving a differentiable optimal transport problem, whose costs are predicted by a graph neural network. We introduce a flexible context aggregation mechanism based on attention, enabling SuperGlue to reason about the underlying 3D scene and feature assignments jointly. Compared to traditional, hand-designed heuristics, our technique learns priors over geometric transformations and regularities of the 3D world through end-to-end training from image pairs. SuperGlue outperforms other learned approaches and achieves state-of-the-art results on the task of pose estimation in challenging real-world indoor and outdoor environments. The proposed method performs matching in real-time on a modern GPU and can be readily integrated into modern SfM or SLAM systems. The code and trained weights are publicly available at github.com/magicleap/SuperGluePretrainedNetwork.

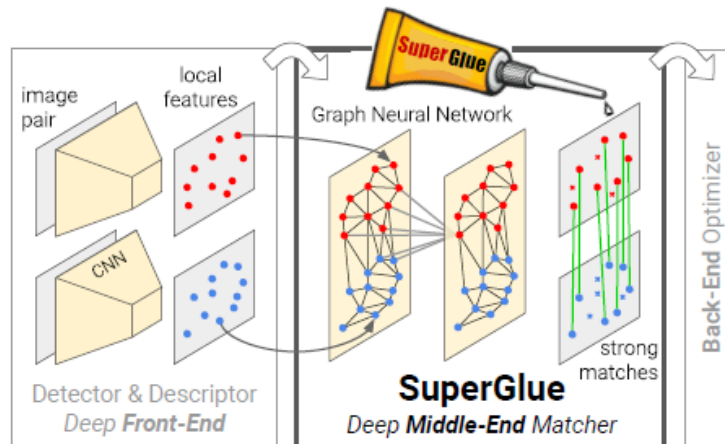


Figure 1: **Feature matching with SuperGlue.** Our approach establishes pointwise correspondences from off-the-shelf local features: it acts as a middle-end between hand-crafted or learned front-end and back-end. SuperGlue uses a graph neural network and attention to solve an assignment optimization problem, and handles partial point visibility and occlusion elegantly, producing a partial assignment.

<https://github.com/magicleap/SuperGluePretrainedNetwork>
k

LightGlue: Local Feature Matching at Light Speed

Philipp Lindenberger¹ Paul-Edouard Sarlin¹ Marc Pollefeys^{1,2}

¹ ETH Zurich ² Microsoft Mixed Reality & AI Lab

Abstract

We introduce *LightGlue*, a deep neural network that learns to match local features across images. We revisit multiple design decisions of *SuperGlue*, the state of the art in sparse matching, and derive simple but effective improvements. Cumulatively, they make *LightGlue* more efficient – in terms of both memory and computation, more accurate, and much easier to train. One key property is that *LightGlue* is adaptive to the difficulty of the problem: the inference is much faster on image pairs that are intuitively easy to match, for example because of a larger visual overlap or limited appearance change. This opens up exciting prospects for deploying deep matchers in latency-sensitive applications like 3D reconstruction. The code and trained models are publicly available at github.com/cvg/LightGlue.

1. Introduction

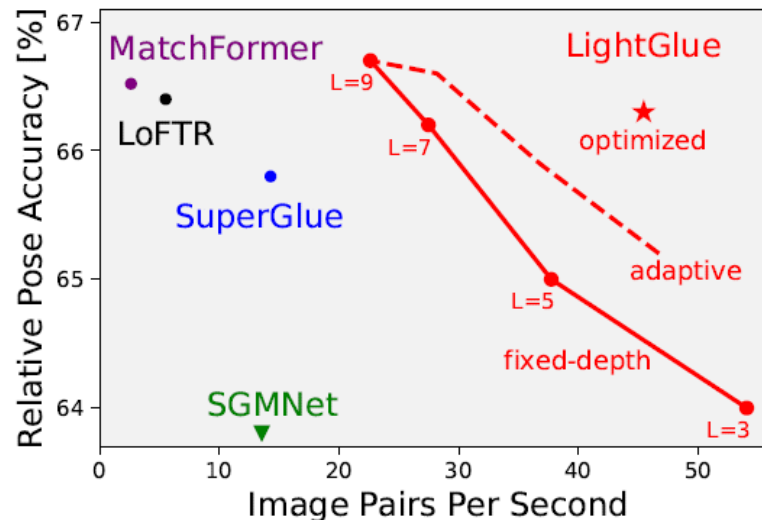


Figure 1. **LightGlue matches sparse features faster and better** than existing approaches like SuperGlue. Its adaptive stopping mechanism gives a fine-grained control over the speed vs. accuracy trade-off. Our final, optimized model ★ delivers an accuracy closer to the dense matcher LoFTR at an $8\times$ higher speed, here in typical outdoor conditions.

<https://github.com/cvg/LightGlue>

OmniGlue: Generalizable Feature Matching with Foundation Model Guidance

Hanwen Jiang*

Arjun Karpur†

Bingyi Cao†

Qixing Huang*

André Araujo†

*University of Texas at Austin

(hwjiang, huangqx)@cs.utexas.edu

†Google Research

(arjunkarpur, bingyi, andrearaujo)@google.com

Abstract

The image matching field has been witnessing a continuous emergence of novel learnable feature matching techniques, with ever-improving performance on conventional benchmarks. However, our investigation shows that despite these gains, their potential for real-world applications is restricted by their limited generalization capabilities to novel image domains. In this paper, we introduce OmniGlue, the first learnable image matcher that is designed with generalization as a core principle. OmniGlue leverages broad knowledge from a vision foundation model to guide the feature matching process, boosting generalization to domains not seen at training time. Additionally, we propose a novel keypoint position-guided attention mechanism which disentangles spatial and appearance information, leading to enhanced matching descriptors. We perform comprehensive experiments on a suite of 7 datasets with varied image domains, including scene-level, object-centric and aerial images. OmniGlue’s novel components lead to relative gains on unseen domains of 20.9% with respect to a directly comparable reference model, while also outperforming the recent LightGlue method by 0.5% relatively. Code and model can be found at <https://github.com/Vincentqyw/omniglue-onnx>.

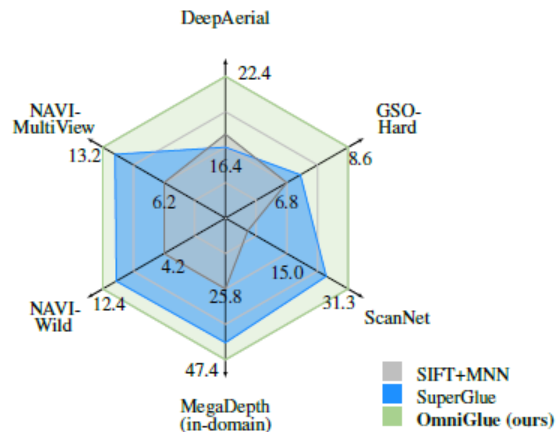


Figure 1. **OmniGlue is a generalizable learnable matcher.** Introducing foundation model guidance and an enhanced attention mechanism, OmniGlue learns effective image matching that transfers well to image domains not seen during training. We compare it against reference methods SIFT [30] and SuperGlue [42], with substantial improvements on a suite of diverse datasets: outdoor scenes (MegaDepth-1500 [26] pose AUC@5°), indoor scenes (ScanNet [8] pose accuracy @5°), aerial scenes (DeepAerial [36] PCK@1%) and object-centric images (GSO-Hard [12] and NAVI-MultiView / NAVI-Wild [19], pose accuracy @5°).

Despite substantial progress, these advancements over

<https://github.com/Vincentqyw/omniglue-onnx>

LoFTR: Detector-Free Local Feature Matching with Transformers

Jiaming Sun^{1,2*} Zehong Shen^{1*} Yuang Wang^{1*} Hujun Bao¹ Xiaowei Zhou^{1†}

¹Zhejiang University ²SenseTime Research

Abstract

We present a novel method for local image feature matching. Instead of performing image feature detection, description, and matching sequentially, we propose to first establish pixel-wise dense matches at a coarse level and later refine the good matches at a fine level. In contrast to dense methods that use a cost volume to search correspondences, we use self and cross attention layers in Transformer to obtain feature descriptors that are conditioned on both images. The global receptive field provided by Transformer enables our method to produce dense matches in low-texture areas, where feature detectors usually struggle to produce repeatable interest points. The experiments on indoor and outdoor datasets show that LoFTR outperforms state-of-the-art methods by a large margin. LoFTR also ranks first on two public benchmarks of visual localization among the published methods. Code is available at our project page: <https://zju3dv.github.io/loftr/>.

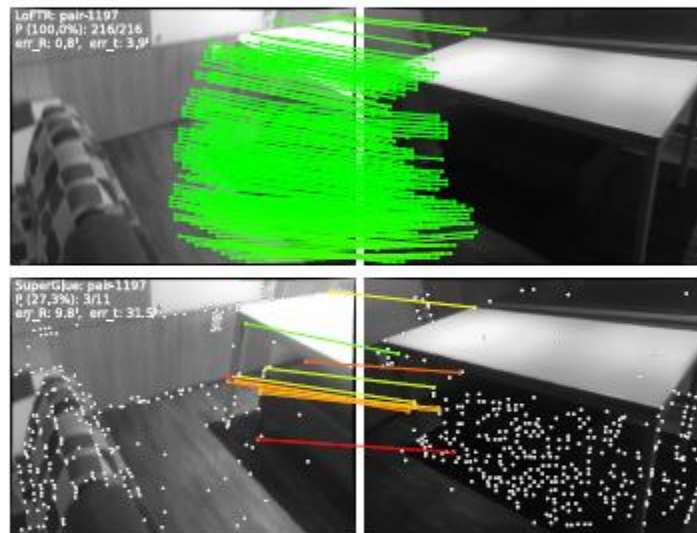


Figure 1: Comparison between the proposed method LoFTR and the detector-based method SuperGlue [37]. This example demonstrates that LoFTR is capable of finding correspondences on the texture-less wall and the floor with repetitive patterns, where detector-based methods struggle to find repeatable interest points.¹

<https://zju3dv.github.io/loftr/>

Homework

- Read one of the proposed papers,
- Evaluate its performances using the available tools,
- Understand the method and associated code,



Write a report with a ppt presentation to explain the method and show matching performances

Group up to 4 people are allowed....